

Практическая работа №6.

Вычисление корней функции одной переменной на языке Python

Цель работы. Целью работы является изучение способов вычисления корней функций одной переменной на Python.

Краткие теоретические сведения

Задача вычисления корней функции одной переменной $f(x)$ является широко распространенной во многих практических приложениях. Корнем (англ. *root*), как известно, является значение аргумента функции, при котором ее значение становится равным нулю.

Обычно необходимо вычислить корни на заданном отрезке изменения аргумента от $x_{\min}$ до $x_{\max}$. Математически изложенное можно записать следующим образом:

$$root = \arg \underset{x}{\{f(x) = 0\}}$$

При этом функция $f(x)$ может быть задана как сравнительно несложным математическим выражением (например, полиномом некоторой степени), так и представлять собой сложный алгоритм преобразования входного аргумента x , настолько сложный, что решить задачу аналитически в этом случае не представляется возможным. Поэтому в общем случае для поиска корней функций используются численные методы, представляющие собой по сути методы подбора значений аргумента, при которых достигается выполнения условия $f(x)=0$. Нужно заметить, что строгое равенство нулю функции, заданной сложной формулой можно обеспечить лишь теоретически, поэтому на практике такой подход может оказаться неработоспособным.

В основу численных методов поиска корней положена довольно очевидная теорема: "Если непрерывная функция на концах некоторого интервала имеет значения разных знаков, то внутри этого интервала у нее есть корень (как минимум, один, но, конечно, может быть и несколько)". Обобщенно этот метод называется дихотомией [см. http://www.machinelearning.ru/wiki/index.php?title=Метод_дихотомии] и представляет собой процедуру деления отрезка пополам с целью сокращения интервала, на котором может находиться корень. Последовательное деление прекращается, когда длина отрезка, на концах которого функция имеет разные знаки, становится меньше, чем заранее установленная разработчиком малая константа ϵ , которая фактически представляет собой абсолютную погрешность найденного значения корня. Таким образом, решение для метода дихотомии можно записать в виде:

$$root = \frac{x_{up} + x_{dn}}{2}, \text{ если } f(x_{dn}) \cdot f(x_{up}) < 0 \text{ и } |x_{up} - x_{dn}| < \varepsilon$$

Нужно заметить, что существуют и другие методы поиска корней (см. https://en.wikipedia.org/wiki/Root-finding_algorithm на англ. яз.), которые различаются вариантами условий постановки задачи, наличием или отсутствием гарантии сходимости к решению в общем случае. Достоинством метода дихотомии является его гарантированная сходимость.

В достаточно развитых языках программирования, к которым относится и Python, существуют модули, содержащие готовые к применению функции и методы вычисления корней функций. В большинстве практически значимых случаев они обеспечивают стабильное решение, хотя опыт показывает, что в некоторых случаях выдаваемые ими результаты вызывают недоумение.

В качестве примера рассмотрим задачу нахождения корней полинома. Эта проблема исследована математиками весьма подробно и для этого частного случая написаны методы и функции, позволяющие находить не только вещественные, но и комплексные корни. В качестве примера в листинге 6.1 приведен текст программы, в котором осуществляется вычисление корней полинома вида

$$f(x) = x^3 - 2.1x^2 + 3.5 \quad (6.1)$$

посредством вызова метода `roots()` из модуля `numpy`. Для работы метода в него нужно передать (списком) коэффициенты полинома при всех степенях x от младшей (всегда нулевой) до старшей (в данном случае – третьей). При задании коэффициентов полинома в списке важно помнить, что отсутствующие члены в формуле для полинома соответствуют умножению данной степени x на коэффициент, равный нулю.

Листинг 6.1. Программа, осуществляющая вычисление корней полинома (6.1)

```
import numpy as np
# сделаем функцию, которая считает полином
# для передаваемого ей списка коэффициентов:
def f(x, coeffs):
    # зеркалим порядок следования коэффициентов,
    # т.к. суммируем начиная с 0-й степени x в цикле for
    # и превращаем список coeffs в массив c:
    c = np.array(coeffs[::-1])
    return np.sum([p*(x**i) for i, p in enumerate(c)])

# зададим коэффициенты полинома
# в естественном для нас порядке от старших степеней к младшим:
coefs = [1, -2.1, 0, 3.5]
```

```

roots = np.roots(coefs)
# выведем результаты:
for i in roots:
    print('корень x=', i, 'f(x)=', f(i, coefs))

```

Для контроля правильности выполнения программы в её конце выводятся пары чисел: значение корня и вычисленное при данном значении аргумента значение функции (которое, естественно, ожидается равным нулю). В результате выполнения программы в данном случае в консоли выведутся следующие сообщения (напомним, что j обозначает мнимую единицу):

```

корень x= (1.5767531849745262+0.9143787191798228j) f(x)= (-3.1086244689504383e-15-5.329070518200751e-15j)
корень x= (1.5767531849745262-0.9143787191798228j) f(x)= (-3.1086244689504383e-15+5.329070518200751e-15j)
корень x= (-1.053506369949054+0j) f(x)= (1.7763568394002505e-15+0j)

```

В данном случае вывод чисел производится в т.н. экспоненциальной форме (запись XeY в ней означает число, равное $X \cdot 10^Y$), откуда легко убедиться, что точного равенства $f(x)=0$ при значениях аргумента x , равного значению корня, не обеспечивается. При выполнении расчетов подобного плана с вещественными числами в Python (да и на других языках программирования) всегда нужно иметь в виду, что числа, меньшие по модулю, чем 10^{-14} , являются т.н. “машинным нулем” и должны восприниматься как 0.

При анализе получаемых результатов всегда полезно помнить, что количество корней полинома всегда равно величине его старшей степени (в данном случае – три), причем в случае нечетных порядков полинома как минимум один из корней всегда будет вещественным (не комплексным) – это видно по величине третьего корня (его мнимая часть равна нулю). Также при анализе получаемых результатов бывает полезно строить график функции, чтобы убедиться, что найденные значения корней не противоречат поведению функции.

Более общим и более сложным является случай, когда функция, у которой нужно найти корни, не является полиномом. В этом случае количество корней может быть ничем не ограничено (например, если в формуле для $f(x)$ присутствует гармоническая функция). В подобных случаях смысл имеет лишь задача отыскания только вещественных корней на заданном интервале изменения аргумента функции.

Рассмотрим пример вычисления корней функции

$$f(x)=3\cos(x)+x^2\cdot\sin(x) \quad (6.2)$$

Программа, которая реализует вычисление корней с использованием функции `fsolve()`, входящей в библиотеку `scipy` (популярнейшая библиотека для выполнения научных расчетов), приведена в листинге 6.2.

Для работы функции `fsolve()` нужно задать начальное приближение к корню, после чего функция будет искать соответствующее значение в окрестности указанного аргумента. Отсюда следует и основной недостаток данного подхода: чтобы вычислить каждый из корней уже каким-то образом необходимо иметь информацию о его приближенном значении. В программе листинга 6.2 совокупность начальных приближений была задана последовательностью чисел, сформированной посредством вызова метода `linspace` из библиотеки `numpy`.

Листинг 6.2. Программа, осуществляющая вычисление корней функции (2)

```
import numpy as np
from scipy.optimize import fsolve
import fplot

def f(x):
    return 3 * np.cos(x) + x**2 * np.sin(x)

(x_min, x_max) = (-10, 10)
intervals = 10
x = np.linspace(x_min, x_max, intervals)
roots = [] # пустой список из корней
for xi in x:
    root = fsolve(f, xi) # ищем корень на очередном интервале
    roots.append(root[0]) # добавляем найденный корень в список
# roots = np.unique(np.round(roots, 2))
# выведем результаты:
num = 0
for xi in roots:
    num += 1
    print(num, 'корень x=', xi, '    f(x)=', f(xi))
fplot.plot(f, x_min, x_max) # строим график функции
```

Очевидно, что непосредственное применение функции при таком подходе приведет к тому, что количество найденных корней окажется равным количеству аргументов, передаваемых в функцию (в данном примере их 10: `intervals = 10`); при этом, если корней у функции по факту меньше, то некоторые из них окажутся задублированы в итоговом ответе, т.к. один и тот же корень окажется ближайшим к нескольким значениям переданного начального приближения, а если количество интервалов сделать слишком малым, то некоторые корни будут пропущены. В данном случае результатом работы функции будет следующее сообщение к консоли:

```
1 корень x= -9.458300148276187  f(x)= 4.574118861455645e-14
2 корень x= -6.357279284165466  f(x)= 1.199040866595169e-14
3 корень x= -6.357279284165466  f(x)= 1.199040866595169e-14
4 корень x= -3.3960698956736968  f(x)= -8.881784197001252e-16
```

5 корень $x = -1.1534047806450516$	$f(x) = -4.218847493575595e-14$
6 корень $x = 1.2324867803642905$	$f(x) = 2.428599740910296$
7 корень $x = 2.768517344573897$	$f(x) = -1.3322676295501878e-15$
8 корень $x = 6.2054353065608545$	$f(x) = -9.769962616701378e-15$
9 корень $x = 2.768517344573897$	$f(x) = -1.3322676295501878e-15$
10 корень $x = 9.39077230259471$	$f(x) = 8.961809072616234e-11$

и еще будет выведено предупреждение:

RuntimeWarning: The iteration is not making good progress, as measured by the improvement from the last ten iterations.

root = fsolve(f, xi) # ищем корень на очередном интервале

Трудно понять причину появления данного сообщения, но легко увидеть, что корень №6 просто не является таковым. Поэтому никогда не рекомендуется игнорировать даже такие предупреждения, которые не вызывают прекращение выполнения программы. Что касается наблюдаемого в данном ответе дублирования найденных корней (попарно одинаковы 2-й и 3-й, а также 7-й и 9-й), то причина этого уже изложена выше и совершенно ясна.

При критическом анализе полученных результатов – а подобный анализ всегда нужно делать – бывает полезно построить график функции (см. рис.6.1).

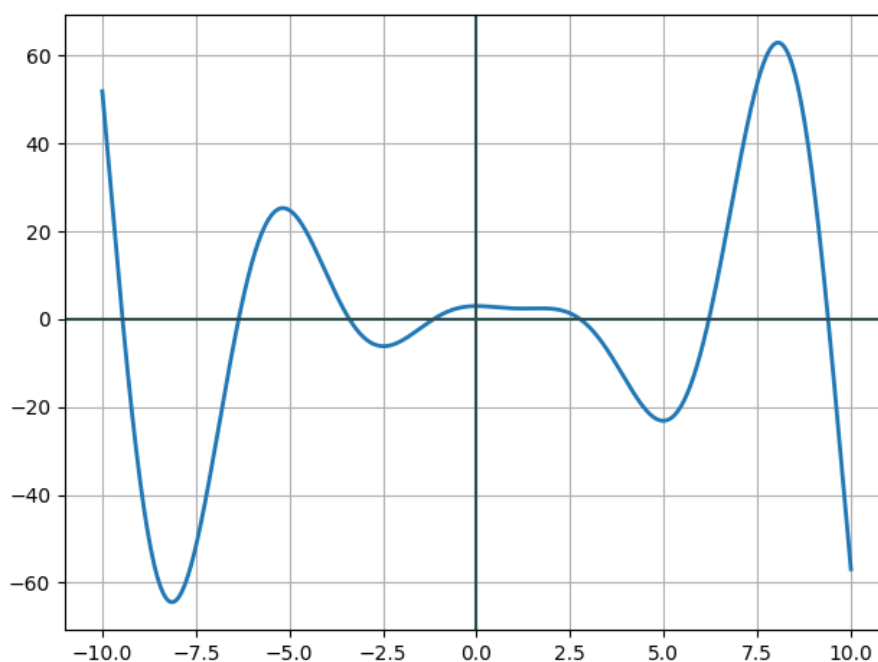


Рис. 6.1. График функции, построенный программой из листинга 6.2

Из рис. 6.1 видно, что корней на интервале изменения аргумента от минус 10 до 10 у функции ровно 7. Конечно, в случаях, когда решается некоторая задача, в которой количество корней невелико, а также и графических зависимостей немного, начальные приближения для корней

можно вводить в программу вручную, считывая необходимые значения из графика “на глазок”. Если заменить в программе листинга 6.2 строчку

```
x = np.linspace(x_min, x_max, intervals)
```

на строчку

```
x = [-9.5, -7, -1.5, -3.5, 2.5, 6.5, 9.5]
```

то результат будет верным и выведен без каких-либо осложнений и предупреждающих сообщений.

1 корень x=	-9.458300148276187	f(x)=	4.574118861455645e-14
2 корень x=	-6.3572792841655055	f(x)=	-1.6084911180769268e-12
3 корень x=	-1.1534047806450503	f(x)=	-3.5083047578154947e-14
4 корень x=	-3.3960698956736968	f(x)=	-8.881784197001252e-16
5 корень x=	2.768517344573683	f(x)=	1.3291590050812374e-12
6 корень x=	6.205435306560863	f(x)=	3.2462921240039577e-13
7 корень x=	9.390772302595908	f(x)=	-1.5261125696497402e-11

Однако, к любой программе, которая содержит как верные ответы, так и некорректные данные, можно приделать “костыли” – дополнительные фильтры, которые будут убирать лишние компоненты, основываясь на их свойствах. В данном случае нетрудно это проделать, поставив в цикле условие на проверку диапазона (не в данном примере, но в общем случае ближайший корень может найтись и за пределами заданного диапазона изменения аргумента x), а также на то, является ли данное значение корнем (проверив условие $|f(x)| < \epsilon$), и не является ли оно *практически* совпадающим с предыдущим (для этого список нужно предварительно отсортировать). В последнем случае для решения задачи можно было бы предложить возможность преобразования списка корней в множество, но следует помнить, что вещественные объекты (типа `float`) могут иметь незначительные различия на уровне машинного нуля.

Фрагмент программы, в котором реализован данный фильтр проверок результатов, полученных программой из листинга 6.2, приведен в листинге 6.3.

Листинг 6.3 – Фильтр, удаляющие некорректные данные

```
roots = list(filter(lambda x: f(x) < 10e-10 \
                        and x>=x_min \
                        and x<=x_max, \
                        roots))
roots = np.unique(np.round(roots, 14))
```

В данном фрагменте кода используются два метода: первый основан на вызове встроенной функции фильтрации `filter()`, аргументом которой является лямбда-функция, проверяющая элементы списка на соответствие всем трем условиям, второй использует метод `unique` из модуля `numpy`, который исключает из списка значения, отличающиеся друг от друга на величину, меньшую чем 14 знаков после запятой. Нужно заметить, что в некоторых случаях найденные корни будут отличаться на бОльшую величину, так что константу 14, возможно нужно будет уменьшить. Однако при этом до соответствующего знака после запятой будут округлены значения корней в итоговом списке `roots`, что может быть нежелательным. Впрочем, в случае крайней необходимости всегда можно написать свой цикл `for`, в котором будут последовательно перебираться элементы списка и формироваться новый, элементы которого будут передаваться в выходной без округления значений.

Нужно заметить, что проблема вычисления корней функции в общем случае очень сложна: всегда можно искусственно *придумать* такую функцию, особенности поведения которой будут затруднять определение всех ее корней в автоматическом режиме даже при помощи самого продвинутого программного кода (например, легко сформировать полином, чтобы его корни, которые нужно отыскать, в самом деле отличалась бы на величину машинного нуля), но к счастью, на практике функциональные зависимости с подобными свойствами не встречаются. Чтобы прочувствовать проблему в качестве иллюстрации можно предложить упражнение на вычисление *всех* корней кажущихся сравнительно простыми на вид функций

$$y_1(x) = \sin(10/|x|) \quad (6.3)$$

и

$$y_2(x) = \sin(4x^3) \quad (6.4)$$

на интервале изменения x от 0.01 до 5 (подобные им функции фигурировали в теоретической части метод. указаний к практической работе №4), с обязательной проверкой того, что в середине интервала между каждой парой найденных корней функция по модулю близка к 1, а знаки функции при этом чередуются при переходе от одного интервала к другому, поскольку основой (6.3) и (6.4) является гармоническая функция.

Варианты заданий

Задание на практическую работу предполагает написание программы на языке Python для вычисления корней заданных функций. При выполнении практической работы нужно выполнить три задания.

1) Задание №1. Написать и запустить на выполнение программу, вычисляющую *все* корни заданного полинома $p(x)$. Формулы для полинома

следует брать из столбца $y_1(x)$ таблицы 4.1 в описании задания на практическую работу №4), также следует построить график $p(x)$ и обязательно нанести на ось абсцисс цветные точки, соответствующие *только вещественным* найденным корням (при этом диапазон изменения аргумента x при построении графика выбрать таким, чтобы в интервал попали все вещественные корни, а границы интервала были бы при этом целыми числами). В случае, если построенный график не позволит наглядно продемонстрировать тот факт, что каждый корень лежит точно на пересечении графика и оси абсцисс, следует построить дополнительный график, используя ключ 'symlog' для оси ординат.

2) Задание №2. Написать и запустить на выполнение программу, вычисляющую все *только вещественные* корни заданной функции $f(x)$ на заданном интервале изменения аргумента x (см. табл 6.1) с заданным значением параметра w , построить график $f(x)$ и нанести на ось абсцисс цветные точки, соответствующие всем найденным корням.

3) Задание №3: построить график в формате контурной плоскости для функции двух аргументов, заданной как $z(x,y)=|p(x+jy)|$, где j – мнимая единица. Таким способом можно наглядно графически отобразить значения не только вещественных, но и комплексных корней полинома, что является довольно познавательным упражнением. Для лучшей визуализации результатов в этом случае рекомендуется применить логарифмический масштаб по оси z .

Таблица 6.1. Данные вариантов заданий*

Вар.	Функция $f(x)$	Вар.	Функция $f(x)$
1	$w \cdot \sin(x) + x^2 \cdot \cos(x)$, $w=8$	16	$w \cdot \cos(x) + x^2 \cdot \sin(x)$, $w=9$
2	$w \cdot \cos(x) + 2x^2 \cdot \sin(x)$, $w=3$	17	$x \cdot \cos(wx) \cdot \exp(-x)$, $0 \leq x \leq 3$, $w=5$
3	$w \cdot \cos(x) + 12x^2 \cdot \sin(x)$, $w=4$	18	$\sin(wx) \cdot \exp(-x)$, $0 \leq x \leq 3$, $w=5$
4	$5 \cos(x) + wx^2 \cdot \sin(x)$, $w=10$	19	$x \cdot \cos(wx^2)$, $-5 \leq x \leq 5$, $w=1$
5	$6 \cos(x) + wx^2 \cdot \sin(x)$, $w=5$	20	$w \cdot \cos(2x) + x^2 \cdot \sin(x)$, $w=7$
6	$7 \cos(x) + wx^2 \cdot \sin(x)$, $w=4$	21	$x \cdot \cos(10x) + w \cdot \sin(4x)$, $0 \leq x \leq 5$
7	$w \cdot \cos(x) + 3x^2 \cdot \sin(x)$, $w=8$	22	$\sin(\pi x) \cdot \exp(-wx)$, $0 \leq x \leq 10$, $w=0.3$
8	$w \cdot \cos(x) + 2x^2 \cdot \sin(x)$, $w=9$	23	$x \cdot \cos(wx^2)$, $-5 \leq x \leq 5$, $w=1$
9	$\cos(\pi x) + wx^2 \cdot \sin(x)$, $w=3$	24	$w \cdot \cos(x) + x^2 \cdot \sin(2x)$, $w=6$
10	$\cos(x) + wx^2 \cdot \sin(\pi x)$, $w=3$	25	$w \cdot \sin(x) + x^2 \cdot \cos(x)$, $w=4$
11	$\cos(\pi x) + x^2 \cdot \sin(wx)$, $w=1$	26	$w \cdot \sin(2x) + x^2 \cdot \cos(x)$, $w=2$
12	$\cos(wx) + x^2 \cdot \sin(\pi x)$, $w=1$	27	$w \cdot \cos(x) + x^2 \cdot \sin(x)$, $w=3$
13	$w \cdot \cos(\pi x) + x^2 \cdot \sin(x)$, $w=3$	28	$w \cdot \sin(x) + x^2 \cdot \cos(x)$, $w=7$
14	$\cos(wx) + x^2 \cdot \sin(wx)$, $w=2$	29	$w \cdot \cos(x) + x^2 \cdot \sin(x)$, $w=7$
15	$\cos(x) + x^2 \cdot \sin(wx)$, $w=2$	30	$w \cdot \cos(x) + x^2 \cdot \sin(3x)$, $w=9$

* **Примечание:** во всех вариантах, где не указан диапазон для переменной x , принять диапазон $-8 \leq x \leq 8$.

При желании обучающегося можно попытаться выполнить дополнительное задание: решить задачу вычисления всех вещественных корней функций, заданных формулами (6.3) и (6.4) в указанном в табл. 6.1 интервале изменения аргумента x , описать возникшие при этом проблемы и/или предпринятые программные решения, а также выявленные закономерности. Для верификации полученных результатов можно, например, построить график зависимости длины интервала между соседними корнями Δx (или обратной ему величины, т.е. $1/\Delta x$) от значения x . Характер соответствующей зависимости нетрудно предвидеть, исходя из свойств аргумента в рассматриваемых тригонометрических функциях.

Порядок выполнения работы

1. Получить вариант задания у преподавателя и убедиться в правильном понимании задания и критериев его оценивания.
2. Используя приведенные в метод. указаниях шаблоны программ, написать и отладить программы, обеспечивающие выполнение двух заданий на данную практическую работу, запустить их на выполнение в IDE и сохранить полученные результаты.
3. При желании попытаться выполнить предложенное дополнительное задание.
4. Получить результаты работы программ и подвергнуть их критическому анализу.

Требования к содержанию отчета

1. Цель работы.
2. Подробные формулировки полученных заданий.
3. Рекомендуется для каждой задачи в отчете отвести свой раздел, в каждом из которых подробно изложить соображения по её решению.
4. Программы с необходимыми комментариями, а также описание наиболее важных их фрагментов в основном тексте отчета.
5. Полученный список из найденных корней с проверкой значения функции в данных точках для двух полученных функций ($p(x)$ и $f(x)$)
6. Полученные графики функций $p(x)$ и $f(x)$ с нанесенными на оси абсцисс точками, соответствующими значениям всех найденных программой вещественных корней.
7. При желании – описание процесса выполнения и результатов для дополнительного задания.
8. Выводы по работе.
9. Список использованной литературы.

При оформлении отчета о практической работе следует придерживаться требований правил, размещенных на сайте ГУАП.

Рекомендуемая литература

1. Python. Сборник упражнений / Стивенсон Б., пер. с англ. А.Ю. Гинько. – М.: ДМК-Пресс, 2021. – 238 с.
2. «Укус Питона» – «A Byte of Python» по-русски / Swaroop С.Н., пер. с англ. В. Смоляр. ver.2.02, apr 2020. – Sphinx. // свободно распространяется по лицензии Creative Commons Attribution-Share Alike 3.0 Unported, 2020.
3. Решаем задачи на Python / Джеймс Дэвис – Изд-во ЛитРес, 2024. – 158 с.
4. Введение в язык программирования Python / Чернышев С.А. – СПб: Изд-во ГУАП, 2023. – 236 с.